

به نام خدا

ایلیا جاهد

پروژه اول : درخت تصمیم

تمامی مراحل پیاده سازی در گیت هاب شخصی قرارداد شده است و در این داکيومنت توضیحات صرفاً روی ورژن نهایی انجام میپذیرد.

توضیحات در ادامه بدین شکل خواهد بود:

1. توضیح عملکرد بلاک کد های مهم به همراه اسکرین شات شان.
2. هر بلاک کد که در کلاس زیاد مورد بحث نبود ، بیشتر تشریح میشود.
3. هر بلاک کد در پیاده سازی اش با چالش مواجه شدم ، توضیح داده میشود.
4. درخت تصمیم نهایی در فایل پی دی اف جداگانه در فایل زیپ فرستاده شده موجود است.
5. سعی میشود برای جلوگیری از حجیم شدن گزارش کار در کد نهایی کامنت هم قرار داده شود تا جزئیات کد در هنگام ارائه توضیح داده شود.

طبیعتاً اولین مرحله در این پروژه پیاده سازی توابعی همچون `entropy` و در ادامه اش `information gain` همچنین `gini` و در ادامه اش `gini split` میباشد:

```
def entropy(y):
    if len(y) == 0:
        return 0.0
    yes_counts = (y == 1).sum()
    total_counts = len(y)
    fraction = yes_counts / total_counts
    if fraction == 0 or fraction == 1:
        return 0
    result = -fraction * log2(fraction) - (1 - fraction) * log2(1 -
fraction)
    return result
```

```
def gini(y):
    if len(y) == 0:
        return 0.0
    total_count = len(y)
    yes_counts = (y == 1).sum()
    fraction = yes_counts / total_count
    if fraction == 0 or fraction == 1:
        return 0
    result = 1 - (fraction ** 2 + (1 - fraction) ** 2)
    return result
```

این دو submethod برای تعیین انترپپی و `gini` برای label های یک `dataframe` میباشد. (چون خروجی `dataset` باینری هست ، تابع به ساده ترین شکل ممکن پیاده شد).

در حقیقت شهودی که این توابع میدهند میزان بی نظمی موجود در label های هر `dataframe` هست که جلوتر برای انتخاب بهترین فیچر به درد خواهد خورد. (که متعاقباً توضیح داده خواهد شد).

```

def _information_Gain(self, feature, X, Y):
    if ((isinstance(Y, pd.Series)) and (Y.name is None)):
        Y = Y.rename("target")
    if isinstance(Y, np.ndarray):
        Y_np_local = Y.copy()
    elif not isinstance(Y, np.ndarray):
        Y_np_local = Y.map({"satisfied": 1, "dissatisfied": 0})
        Y_np_local = Y.to_numpy()

    def entropy(y):
        if len(y) == 0:
            return 0.0
        yes_counts = (y == 1).sum()
        total_counts = len(y)
        fraction = yes_counts / total_counts
        if fraction == 0 or fraction == 1:
            return 0
        result = -fraction * log2(fraction) - (1 - fraction) * log2(1 - fraction)
        return result

    desired_column = X[feature]
    parent_entropy = entropy(Y_np_local)

    if (pd.api.types.is_numeric_dtype(desired_column) and len(desired_column.dropna().unique()) >= 5):
        best_gain = float("-inf")
        best_threshold = None
        sortedserie = np.sort(desired_column.dropna().unique())
        candidate_thresholds = (sortedserie[:-1] + sortedserie[1:])/2
        for threshold in candidate_thresholds:
            left_part = desired_column <= threshold
            right_part = desired_column > threshold
            left_y = Y_np_local[left_part.to_numpy()] if isinstance(left_part, pd.Series) else Y_np_local[left_part]
            right_y = Y_np_local[right_part.to_numpy()] if isinstance(right_part, pd.Series) else Y_np_local[right_part]
            if (len(left_y) == 0 or len(right_y) == 0):
                continue
            weight_left = len(left_y) / len(Y_np_local)
            weight_right = len(right_y) / len(Y_np_local)
            gain = parent_entropy - (weight_left * entropy(left_y) + weight_right * entropy(right_y))
            if gain > best_gain:
                best_gain = gain
                best_threshold = threshold
        if best_threshold is None:
            return 0, None
        return best_gain, best_threshold
    else:
        desired_narray = desired_column.to_numpy()
        merged = np.column_stack((desired_narray, Y))
        subentropies_sum = 0
        for value in pd.unique(desired_narray): # <- changed here: it works both with Pandas Series and NumPy arrays
            filtered_np = merged[merged[:, 0] == value]
            weight = len(filtered_np) / len(merged)
            subentropies_sum += weight * entropy(filtered_np[:, 1])
        return float(parent_entropy - subentropies_sum), None

```

این بلاک کد یک متود برای تعیین information gain برای یک فیچر مشخص است که در بخش getbest feature از این متود استفاده خواهد شد.

این تابع بعد از چندین ورژن اپدیت به این مرحله رسیده. (مواردی که هی اپدیت داده شد اولاً به خاطر این بود که طرز ساخت درخت در مواجهه با یک فیچر categorical با یک فیچر numerical متفاوت است. برای همین خروجی این تابع به شکل تاپل است.)

بلاک کد بالا نیاز به توضیح بیشتر دارد : فیچر های دیتاستی که داریم ممکن است عددی باشند که به اصطلاح **numerical features** میگویند. این فیچر ها چون مقادیر یونیک زیادی دارند ، در تابع **create tree** که جلوتر مورد بحث قرار میگیرد، اگر از حالت **threshold** استفاده نکنیم **childnode** های خیلی زیادی قرار است ساخته شود که درختمان در این صورت غیر اوپتیمال میشود. پس برای جلوگیری از این اتفاق ما از **threshold** استفاده کردیم و گفتیم یک ارایه از **threshold** های ممکن درست کن و بعد همه ی این **threshold** ها رو یکی یکی بررسی کن. اون **threshold** ای رو انتخاب کن که بیشترین **info gain** را میدهد و در یک متغیر به اسم **best threshold** سیو کن مقدارش رو. (در مورد نحوه ی محاسبه ی **info gain** هم که **entropy** به دست آمده از **label** های دیتافریم اصلی را از جمع وزنی انتروپی **label** های زیردیتا فریم کم میکنیم).

در مواردی هم که فیچر مون **categorical** بود یعنی یک تعداد جواب مشخص داشت ، در این صورت **threshold=None** برمیگردانیم.

دو جا در کد بالا **comment** زده ام. این ها مربوط میشود به چالشی که داشتم. یعنی قبلا سینتکسی که زده بودم فقط برای نامپای کار میکرد ولی در این سینتکس برای **pandas serie** هم کار میکند. (نسخه ی قبلی در گیت هاب موجود است).

(این مشکل در مراحل آخر پروژه یعنی **train** کردن مشخص شد که مجبور شدم برای هندل کردن مشکل کد از **chatgpt** جاهایی که **convert** بین **pandas dataframe** , **pandas serie** , **numpy array** هست رو برایم مشخص کنه. چون تعداد سطر های کد زیاد شده بود و عملا بررسی دستی این موارد خیلی وقت گیر بود).

```

def _gini_Split(self, feature, X, Y):
    if ((isinstance(Y, pd.Series)) and (Y.name is None)):
        Y = Y.rename("target")
    if isinstance(Y, np.ndarray):
        Y_np_local = Y.copy()
    elif not isinstance(Y, np.ndarray):
        Y_np_local = Y.map({"satisfied": 1, "dissatisfied": 0})
        Y_np_local = Y_np_local.to_numpy()

    def gini(y):
        if len(y) == 0:
            return 0.0
        total_count = len(y)
        yes_counts = (y == 1).sum()
        fraction = yes_counts / total_count
        if fraction == 0 or fraction == 1:
            return 0
        result = 1 - (fraction ** 2 + (1 - fraction) ** 2)
        return result

    desired_column = X[feature]

    if (pd.api.types.is_numeric_dtype(desired_column) and len(desired_column.dropna().unique()) >= 5):
        best_score = float("inf")
        best_thr = None
        sortedserie = np.sort(desired_column.dropna().unique())
        candidate_thresholds = (sortedserie[:-1] + sortedserie[1:]) / 2
        for threshold in candidate_thresholds:
            left_part = desired_column <= threshold
            right_part = desired_column > threshold
            left_y = Y_np_local[left_part.to_numpy()] if isinstance(left_part, pd.Series) else Y_np_local[left_part]
            right_y = Y_np_local[right_part.to_numpy()] if isinstance(right_part, pd.Series) else Y_np_local[right_part]
            if (len(left_y) == 0 or len(right_y) == 0):
                continue
            weight_left = len(left_y) / len(Y_np_local)
            weight_right = len(right_y) / len(Y_np_local)
            score = weight_left * gini(left_y) + weight_right * gini(right_y)
            if score < best_score:
                best_score = score
                best_thr = threshold
        if best_thr is None:
            return float("inf"), None
        return best_score, best_thr
    else:
        desired_narray = desired_column.to_numpy()
        merged = np.column_stack((desired_narray, Y_np_local))
        Sub_gini_sum = 0
        for value in pd.unique(desired_narray): # <- changed here: it works both with Pandas Series and NumPy arrays
            filtered_df = merged[merged[:, 0] == value]
            weight = len(filtered_df) / len(merged)
            Sub_gini_sum += weight * gini(filtered_df[:, 1])
        return float(Sub_gini_sum), None

```

این مرحله هم مشابه توضیحات بلاک کد قبلی است. (در صورت وجود هرگونه سوال، در زمان ارائه پاسخگو هستیم.)

حالا وارد مرحله ی دوم پروژه میشویم: یعنی اینکه بتوانیم با یک ورودی دیتافریمی از کل فیچر ها با داده هایش، یک ورودی پانداس سری از label های corresponding این دیتافریم، به بهترین فیچر دست بیابیم. توضیح جزئیات این مرحله در صفحه ی بعد:

```

# X is dataframe of features,Y is series of labels.
def _get_best_Feature(self, X, Y):
    if isinstance(X, np.ndarray):
        X = pd.DataFrame(X)
    if isinstance(Y, pd.Series):
        local_Y = Y.copy()
    elif isinstance(Y, np.ndarray):
        local_Y = pd.Series(Y)
    else:
        local_Y = pd.Series(Y)

    local_Y = local_Y.reset_index(drop=True)

    best_feature = None
    best_threshold = None
    gain = None
    gini = None

    if self.mode == "gain":
        max_val = float('-inf')
        for feature in list(X.columns):
            info_gain, threshold = self._information_Gain(feature, X, local_Y)
            if info_gain > max_val:
                max_val = info_gain
                best_feature = feature
                best_threshold = threshold
        gain = max_val

    elif self.mode == "gini":
        min_val = float('inf')
        for feature in list(X.columns):
            gini_score, threshold = self._gini_Split(feature, X, local_Y)
            if gini_score < min_val:
                min_val = gini_score
                best_feature = feature
                best_threshold = threshold
        gini = min_val

    if best_feature is None:
        return Node(feature=None, datasample_labels=Y, children=[])

    if best_threshold is not None:
        return Node(feature=best_feature, datasample_labels=Y, children=[f"<={best_threshold}", f">{best_threshold}"], threshold=best_threshold, gain=gain, gini=gini)
    else:
        bestfeature_series = X[best_feature].reset_index(drop=True)
        merged_with_label = pd.concat([bestfeature_series, local_Y], axis=1)
        children_list = []
        for uniqueval in merged_with_label[best_feature].unique().tolist():
            children_list.append(uniqueval)
        return Node(feature=best_feature, datasample_labels=Y, children=children_list, gain=gain, gini=gini)

```

همانطور که مشخص است در متود get best feature قرار است یک object از نوع node برگردانیم.

این object شامل اطلاعات مهمی است که در هنگام ساختن درخت و همچنین traverse کردن درخت به دردمان خواهد خورد که شامل این موارد هستند:

Best feature , children ,gain and gini, satasample_labels

اینکه چرا gain و gini هم در ابجکت ما هست مربوط میشود به pruning threshold که این مورد هم جزو چالش های من بود. چون این هایپر پارامتر را در اواخر پروژه اضافه کردم و کدی که داشتم هم پیچیده شده بود و وقت زیادی گرفت. در مورد جزئیات این چالش در متود create tree توضیح خواهم داد.

یک نکته ای هم که در این بلاک کد بهتر بود که توضیح بدهم ، reset index کردن است. در غیر این صورت همه ی merge کردن ها برای فیچر و corresponding label ها همگی بهم میخورد. این مورد رو پس از دیباگ کردن متوجه شدم. کد مربوط به دیباگ کامنت شده و در نسخه های قبلی در گیت هابم مشخص است.

یک نکته ی دیگر هم این که ما در این مرحله children مربوط به یک node را به صورت ارایه ای از stringها در node ذخیره میکنیم. شاید این کار باعث شد که بعدا در پیاده سازی متود create tree با چالش های زیادی مواجه بشوم ولی این کار بنظرم پیاده سازی این متود را خیلی ساده تر کرد.

مرحله ی سوم پیاده سازی متودی برای ساخت درخت است که در واقع ، بنیادی ترین بخش پروژه همین بخش است:

```
#For assigning the Root to the real root node in this tree:
def fit(self, X_Train, Y_Train):
    # Defensive: convert numpy to DataFrame if needed
    if isinstance(X_Train, np.ndarray):
        X_Train = pd.DataFrame(X_Train)
    if isinstance(Y_Train, pd.Series):
        Y_Train = Y_Train.map({"satisfied": 1, "dissatisfied": 0, 1:1, 0:0})
        Y_Train = Y_Train.to_numpy()
    self.root = self._create_Tree(X_Train,Y_Train,depth= 0)
    return self.root
```

این بلاک کد همچنان که از commentهایی که نوشتم مشخص است که متودی است که اولش یک سری convertها میکنه ورودی ها رو تا هنگام صدا زدن تابع create tree قابل استفاده باشد[مثلا numpy array باشه convert میشه به pandas dataframe. نکته ی وجود همچین تابعی هم اینه که برای کلاس decision tree میاد خروجی create tree(که در بلاک کد بعدی توضیح داده میشود) به پراپرتی root ، assign میکنه .]

حالا بررسی کد بلاک مربوط به تابع create tree :

```
# Y is the label column and X is the dataframe of the attributes
def _create_Tree(self, X, Y, parent=None, depth=0):

    # helper method for edge cases in recursion (like the given method named :_calculate_Value)
    def plurality_value(Y): # Y is actually a numpy array
        yes_counts = (Y == 1).sum()
        total_counts = len(Y)
        # 1 is interpreted as satisfied and 0 as dissatisfied
        return {1: yes_counts/total_counts, 0: (total_counts-yes_counts)/total_counts} #returning a dict of two probability.

    all_features = list(X.columns)
    num_Samples = len(Y)

    # --- edge cases before splitting ---
    if len(all_features) == 0 or num_Samples == 0 or np.all(Y==1) or np.all(Y==0):
        if len(all_features) == 0 or num_Samples == 0:
            result = plurality_value(Y if num_Samples>0 else parent.datasample_labels)
            probable_label = max(result, key=result.get)
            return Node(isleaf=True, datasample_labels=Y, parent=parent, answer=probable_label)
        if np.all(Y==1):
            return Node(isleaf=True, answer=1, parent=parent, datasample_labels=Y)
        if np.all(Y==0):
            return Node(isleaf=True, answer=0, parent=parent, datasample_labels=Y)

    # Get best feature node
    best_feature_node = self._get_best_Feature(X, Y)
    best_feature_node.parent = parent

    # --- compute x for pruning check ---
    if self.mode == "gain":
        x = best_Feature_node.gain
    else:
        x = best_Feature_node.gini

    # --- main stopping condition including pruning threshold --- # changed
    if num_Samples < self.min_Samples or depth >= self.max_Depth or (self.pruning_threshold is not None and ((self.mode=="gain" and x <= self.pruning_threshold) or (self.mode=="gini" and x >= self.pruning_threshold))):
        result = plurality_value(Y)
        probable_label = max(result, key=result.get)
        return Node(isleaf=True, answer=probable_label, datasample_labels=Y, parent=parent)

    best_Feature_node.children = []

    current_threshold = best_Feature_node.threshold

    if current_threshold is not None:
        # ---
    else:
        # ---
```

اولا این کد رو سعی کردم مطابق سودوکد موجود در اسلاید های تدریس شده استاد پیاده سازی کنم. نکته ی اول اینکه این تابع recursive هستش پس طبیعتا باید base case هایی داشته باشه. به عنوان نمونه اگر همه ی label های یک دیتافریمی (که اینجا ورودی Y هستش) مثلا 1 باشه (satisfied مپ شده به 1 و dissatisfied هم مپ شده به 0) میاد به leafnode. برای درک کامل leafnode و سایر موارد مربوط به node بهتر است کلاس node رو ببینید:

```
class Node():
    def __init__(self, feature=None, children=None, isleaf = False, parent=None, datasample_labels = None, answer=None, edge_value= None, threshold = None, gain=None, gini=None): # changed: added gain and gini attributes
        # Initialize node attributes
        self.feature = feature
        self.datasample_labels = datasample_labels
        self.children = children #children should be given as node
        self.is_leaf = isleaf
        self.parent = parent
        self.answer = answer
        self.edge_value = edge_value #for categorical splits
        self.threshold = threshold #for numeric splits
        self.gain = gain # changed: store gain
        self.gini = gini # changed: store gini
```

(در مورد سایر basecase ها هم عین سودوکد اسلاید تدریسی استاد عمل کرده ام که اگر نیاز باشد در ارائه توضیح میدهم)

حالا بخش اصلی تابع به این شکل هست که با استفاده از getbestfeature method میایم و بهترین فیچر رو در قالب node در یک متغیری ذخیره میکنیم. (همانطور که در بخش های قبلی هم گفته شد این

node صرفا فیچر رو نگه نمیدارد و بقیه موارد همچون children ها رو هم در قالب ارایه ای از استرینگ ها نگه میدارد.)

بعد از این باید parent اش رو مشخص کنیم.

همانطور که در بخش های قبلی به یک چالشی اشاره کردم (اضافه کردن pruning threshold) اینجا توضیحش میدم:

این هایپرپارامتر ما قرار است اگر infogain یا ginisplit برای یک فیچری کم باشد ، از ادامه دادن ساخت tree برای آن نود (نودی که این فیچر رو داره) جلوگیری کند. یعنی در سودو کدی که فرستادم بره به بخش else که هنوز کد بلاکش مشخص نیست و در ادامه میفرستم. یعنی قراره ان نود رو leafnode بکنه (چه شکلی قراره leafnode بشه؟ با true کردن پراپرتی isleaf برای ان نود). علاوه بر این چند تا hyperparameter دیگر هم داریم.

موضوع صحبتیم همین هایپرپارامتر pruning threshold بود. برای اینکه اینکه بتونیم در این متود باهاش کار کنیم طبیعتا node ما باید یک پراپرتی از gain یا gini فیچر مربوط به ان نود داشته باشد و این پراپرتی همون gain, gini ای بود که در مراحل قبل توضیح دادم (خروجی getbestfeature)

نیاز بود که این gini و gain براساس یک معیار باشند که create tree عملکرد نامتعارفی نداشته باشد (منظورم اینه که تابع infogain یا چیزی که خروجی میدهد انتروپی خودش منهای جمع میانگین وزنی بچه هاش ولی ginisplit صرفا جمع میانگین وزنی بچه ها. این موضوع رو در بخش getbestfeature هندلش کرده ام و کاملا مشخص هست از خروجی gini که در انجا گذاشتم. یعنی برای gini هم عملکردی مشابه gini خودش منهای جمع میانگین وزنی بچه هاش اعمال کردم).

حالا نحوه ی کارمون هم ساده تر شد با این gain, gini در ابجکت نودمون. طبق سودو کدی که فرستادم برای create tree میایم به متغیر X رو مقدار دهی میکنیم براساس اینکه این decision tree ما mode اش gini هست یا gain. (حالا با چی مقدار دهی میکنیم؟ با همین gini یا gain که در نود bestfeaturenode گذاشتیم).

حالا میتونیم بریم چک کنیم که اگر شرط های مربوط به هایپرپارامترها مون meet نشدن ، به leafnode بدیم. در غیر اینصورت وقت ساخت tree با استفاده از childnode هست. حالا اگر اون bestnode ما

threshold داشته باشد یک کاری باید انجام بدهیم (چون threshold داشتن یعنی اینکه numerical feature هستش.) و اگر threshold نداشت یک کار دیگر (چون threshold نداشتن به این معناست که categorical feature هستش.)

```
best_Feature_node_children = []

current_threshold = best_Feature_node.threshold

if current_threshold is not None:
    featurename = best_Feature_node.feature
    X = X.reset_index(drop=True)
    local_Y = pd.Series(Y, name='target')
    merged = pd.concat([X, local_Y], axis=1)

    # Separate mask based on threshold
    below_mask = merged[featurename] <= current_threshold
    above_mask = merged[featurename] > current_threshold

    # Sub-DataFrames for features (without current feature)
    below_X = merged.loc[below_mask].drop(columns=[featurename, 'target'])
    above_X = merged.loc[above_mask].drop(columns=[featurename, 'target'])

    # Corresponding labels
    below_Y = merged.loc[below_mask, 'target'].to_numpy()
    above_Y = merged.loc[above_mask, 'target'].to_numpy()

    # Create child nodes recursively
    leftchild = self._create_Tree(below_X, below_Y, parent=best_Feature_node, depth=depth+1)
    rightchild = self._create_Tree(above_X, above_Y, parent=best_Feature_node, depth=depth+1)

    # Assign edge values to children
    leftchild.edge_value = f"<= {current_threshold}"
    rightchild.edge_value = f"> {current_threshold}"

    best_Feature_node_children.append(leftchild)
    best_Feature_node_children.append(rightchild)
    best_Feature_node.children = best_Feature_node_children
    return best_Feature_node
```

این برای حالتی هستش که numerical feature هستش فیچر داخل node مورد بررسی مون:

کاری که انجام میده میاد براساس threshold داده های dataframe رو جدا میکنه. بعدش براساس همونا دو تا child درست میکنه. یکی leftchild و اون یکی rightchild.

موضوع مهم دیگر بحث edge_value هستش که در حقیقت جلوگیری میکند از ساخت یک کلاس دیگر به نام edge. یعنی برای هر childnode مشخص میکنیم با چه value شده این child. مثلاً در حالت numerical، threshold مون میشه edgevalue. در نهایت هر بچه را به bestfeaturenodechildren اضافه میکنم. آخر سر اون آرایه ی استرینگی از بچه ها که ذخیره کرده بود رو تغییر میدم به این متغیر. یعنی الان حتی node ما بچه هایش استرینگ نیست. بلکه چند تا node object واقعی است.

آخر سر return میکنم این bestfeaturenode رو برای استفاده در تابع fit.

برای حالت categorical :

```
else:
    for unique_val in best_Feature_node.children:
        featurename= best_Feature_node.feature
        local_Y = pd.Series(Y,name='target')
        X = X.reset_index(drop=True)
        merged = pd.concat([X,local_Y],axis=1)
        filtered_by_child = merged[merged[featurename]==unique_val]

        if filtered_by_child.empty:
            continue

        new_x = filtered_by_child.drop(columns=[featurename, 'target'])
        new_y = filtered_by_child['target'].to_numpy()

        if len(np.unique(new_y)) == 1:
            leaf = Node(
                isleaf=True,
                answer=new_y[0],
                datasample_labels=new_y,
                parent=best_Feature_node,
                edge_value=unique_val
            )
            best_Feature_node.children.append(leaf)
            continue
        else:
            childnode = self._create_Tree(X=new_x,Y=new_y,parent=best_Feature_node,depth=depth+1)
            childnode.edge_value = unique_val
            best_Feature_node.children.append(childnode)

    best_Feature_node.children = best_Feature_node.children
    return best_Feature_node
```

در این حالت هم قرار است که برای childnode ها subdataframe فیلتر شده براساس مقادیر child که در ارایه ای از استرینگ ها قرار دارد ، childnode بسازیم.همان روندی که در حالت قبل توضیح دادم برای این حالت هم صادق است.صرفا چند تا حالت base دارد که باید دقت کنیم تا از ساختن duplicate node ها جلوگیری کنیم(یکی دیگر از چالش هایی که داشتم).

در مورد basecase ها هم میشه بیشتر صحبت کرد که فعلا برای جلوگیری از حجیم شدن داک توضیحی نمیدهم.

مرحله ی اخر implementation هستش predict (جزو اصلی ترین بخش های این کلاس decision tree)

در این مرحله هدف این است که یک پانداس دیتافریم دریافت کنیم و بهشون label بدیم.این یعنی prediction یا همان پیش بینی:

```
def predict(self, X):
    """
    Predict labels for rows in X.
    Accepts: pandas.DataFrame, pandas.Series (single row), or numpy 2D array.
    Returns: numpy.ndarray of int labels.
    """
    # this is a helper method for plurality, here we have a node and based on the labels it has, we return the value with maximum counts.
    def plurality_label_from_node(node):
        array = node.datasample_labels
        if array is None:
            return 0
        array = np.asarray(array)
        if array.size == 0:
            return 0
        yes_counts = 0
        for x in array:
            if x == 1:
                yes_counts += 1
        no_counts = len(array) - yes_counts
        return 1 if yes_counts > no_counts else 0

    def _move_tree(sample, node):
        # --- input normalization ---
        if self.root is None:
            # no tree fitted -> return zeros
            if X is None:
                return np.array([], dtype=int)
            if isinstance(X, (pd.DataFrame, pd.Series)):
                n = len(X) if isinstance(X, pd.DataFrame) else 1
                return np.zeros(n, dtype=int)
            arr = np.asarray(X)
            if arr.ndim == 1:
                return np.zeros(1, dtype=int)
            return np.zeros(arr.shape[0], dtype=int)
        # convert inputs to a DataFrame of rows
        if isinstance(X, np.ndarray):
            X_proc = pd.DataFrame(X)
        elif isinstance(X, pd.Series):
            X_proc = X.to_frame().T.reset_index(drop=True)
        else:
            X_proc = X.reset_index(drop=True)
        preds = []
        for _, row in X_proc.iterrows():
            sample_dict = row.to_dict()
            try:
                p = _move_tree(sample_dict, self.root)
            except Exception:
                # sometimes fallback to root plurality
                p = plurality_label_from_node(self.root)
            preds.append(int(p))
        return np.array(preds, dtype=int)
```

این کد از همانطور که مشخص است از یک تابع کمکی برای تعیین plurality استفاده کرده است(برای مواقعی که به node ای در درختمون میرسیم که اصلا feature ای ندارد).

حالت کلی کد به این صورت است که قرار است یک ورودی ای که میگیریم برای پیش بینی جوابهاش اولاً باید به صورت dataframe باشد.پس حتما در مرحله ی اول اگر هم به شکل دیتافریم نبود به دیتافریم convert اش میکنیم.همانطور که از هدفمون هم مشخص است (پیش بینی) باید درخت رو پیمایش کنیم.برای پیمایش نیاز است که هر سمپل رو به هر حالتی (که اینجا دیکشنری گرفتیم) تبدیل کنیم و مطابق با اون درخت رو traverse بکنیم.پس یک loop میزنیم و هر بار هر سمپل رو که به دیکشنری تبدیل کردیم ، بعدش میفرستیم به تابع movetree که دقیقاً همین هدف پیمایش درخت و برگرداندن پیش بینی را دارد. در نهایت هر iteration در این loop ای که گفتیم تمام شود یک جوابی از تابع movetree گرفته ایم.این جواب را در آرایه ای ذخیره میکنیم و بعد از تمام شدن loop این آرایه را خروجی میدهیم.همانطور که مشخص

بود در مورد move tree به صورت abstract بحث کردیم. پس در این بخش به توضیح بلاک movetree میپردازیم:

```
def _move_Tree(sample, node):

    # defensive
    if node is None:
        return 0

    if (node.is_leaf):
        return int(node.answer)

    feature = node.feature
    if feature is None:
        return plurality_label_from_node(node)

    # sample may be missing the feature
    if feature not in sample:
        return plurality_label_from_node(node)

    val = sample[feature]

    threshold = node.threshold
    # numeric split (threshold present)
    if threshold is not None:
        # expected children: [left_child, right_child]
        children = node.children
        if not children or len(children) < 2:
            return plurality_label_from_node(node)
        try:
            # handle NaN or non-comparable => fallback
            go_left = (val <= threshold)
        except Exception:
            return plurality_label_from_node(node)

        child = children[0] if go_left else children[1]
        if child is None:
            return plurality_label_from_node(node)
        return _move_Tree(sample, child)

    # categorical split: match child.edge_value to sample value
    children = node.children
    if not children:
        return plurality_label_from_node(node)

    for child in children:
        # child's edge_value should hold the category that leads to that child
        edgevalue = child.edgevalue
        # compare with exact match or string-equal
        if edgevalue == val or (edgevalue is not None and str(edgevalue) == str(val)):
            return _move_Tree(sample, child)

    # unseen category -> plurality fallback
    return plurality_label_from_node(node)
```

این تابع یک سمپل (که گفته شد در توضیحات قبل به شکل دیکشنری است) و یک node را میگیرد و تا زمانی که به یک leafnode یا اینکه یک نود به صورتی که فیچرش none باشد نرسد به صورت

recursive به سمت برگ های درخت می رود تا در نهایت بتواند بر اساس برنج هایی که طی می کند به یک leafnode برسد. پراپرتی answer برای این leafnode که رسیده است را ریترن می کند. این پراپرتی در حقیقت predict مدل از این سمپل است.

الان رسیدیم به بخش implementation ها در notebook :

برای سرعت بیشتر در اجرای بلاک کد ها این نوت بوک را در colab اجرا کردم.

حالا بلاک کد ها تشریح می کنم:

برای بخش hyperparameter tuning چیزی که مدد نظر داشتیم این کد بود:

```
#=====
# Hyperparameter Optimization
#=====
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from DT_Library import

# Loading data
df = pd.read_csv("EuroRail_Survey_cleaned.csv")
X = df.drop("satisfaction", axis=1)
y = df["satisfaction"].map({"satisfied": 1, "dissatisfied": 0})

# Split into train+val (80%) and test (20%) ---
X_trainval, X_test, y_trainval, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Split trainval into train (80%) and validation (20%) ---
X_train, X_val, y_train, y_val = train_test_split(X_trainval, y_trainval,
test_size=0.2, random_state=42)
# So: train=64%, val=16%, test=20%

#Define hyperparameter grid
param_grid = {
    "max_Depth": [10, 12, 14],
    "mode": ["gain", "gini"],
    "min_Samples": [50, 60, 70],
```

```

        "pruning_threshold": [0.01, 0.03, 0.05],
    }

best_acc = -1
best_params = None

print("\nStarting Hyperparameter Optimization...\n")

for mode in param_grid["mode"]:
    for max_d in param_grid["max_Depth"]:
        for min_s in param_grid["min_Samples"]:
            for prune_th in param_grid["pruning_threshold"]:
                tree = DecisionTree(
                    mode=mode,
                    max_Depth=max_d,
                    min_Samples=min_s,
                    pruning_threshold=prune_th
                )

                # Train on train set
                tree.fit(X_train, y_train)

                # Validate on validation set
                y_pred = tree.predict(X_val)

                # Calculate accuracy
                acc = accuracy_score(y_val, y_pred)

                print(f"mode={mode}, max_Depth={max_d}, min_Samples={min_s},
prune_th={prune_th} → acc={acc:.4f}")

                # Track best combination
                if acc > best_acc:
                    best_acc = acc
                    best_params = {
                        "mode": mode,
                        "max_Depth": max_d,
                        "min_Samples": min_s,
                        "pruning_threshold": prune_th
                    }

print("\nBest Hyperparameters:")
print(best_params)
print(f"Validation Accuracy = {best_acc:.4f}")

```

اولا این کد در حقیقت `gridsearch` میباشد یعنی اینکه برای کل حالات ممکن برای مقادیر داده شده برای هایپرپارامترها یک مدل میسازد. این مدل داده ای که باهش `train` همیشه بخش 64 درصدی داده هامون هستش. یعنی اون دیتاستی که داریم اولاً به دو بخش 20 درصدی (داده های تست) و 80 درصدی (داده های `train`) تقسیم میشود. از این داده های `train` ، 25 درصد اش همیشه داده های `validation` و بقیه همیشه داده هایی که قراره در این مرحله مدل مون در هر `iteration` از این `loop` باهش `train` بشود. ضمناً در هر `iteration` این لوپ همانطور که مشخص هستش ، مقادیر هایپرپارامتر های متفاوت داده میشود. یعنی یک `iteration` با `iteration` بعدی مقادیر هایپرپارامتر متفاوت دارد. کار این لوپ این هست که بعد ساخت مدل در هر `iteration` بررسی کند که `accuracy` این مدل روی داده های `validation` چقدر است. بعد در اخر هایپرپارامتر های اون مدلی که بیشترین `accuracy` را دارد به ما بدهد تا بتوانیم در مرحله ی بعد از ان استفاده کنیم. (در بخش `train` و `test` اصلی.)

ولی مشکل اصلی اینجاست و همان دلیلی که من به `colab` ، `switch` کردم. تایم اجرای این کد به خاطر دیتاست بزرگ و اینکه هی قرار هست در این کد `model` بسازیم ، بسیار زیاد بوده و از برای کد من در حدود 10 یا 12 ساعت بود. در `colab` هم با مشکل قطع و وصل مواجه شدم که نتیجتاً طی صحبت هایی که با head TA داشتم ، مجبور بودم که `checkpoint` اضافه کنم به کدم تا در صورت قطعی `gpu` نتیجه تا ان لحظه را در یک فایل نگه دارد و هنگام وصل شدن از ان جایی که مانده شروع به کار کند: (برای این کار از `chatgpt` کمک گرفتم)

```
import os
import joblib
import pandas as pd
from itertools import product
from sklearn.model_selection import train_test_split
from DT_Library import DecisionTree

df = pd.read_csv("EuroRail_Survey_cleaned.csv")
X = df.drop("satisfaction", axis=1)
y = df["satisfaction"].map({"satisfied": 1, "dissatisfied": 0})

X_trainval, X_test, y_trainval, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_trainval, y_trainval,
test_size=0.25, random_state=42)

param_grid = {
```

```

    "max_depth": [10, 12, 14],
    "mode": ["gain", "gini"],
    "min_Samples": [50, 60, 70],
    "pruning_threshold": [0.005, 0.01, 0.02],
}

ckpt_dir = "checkpoints"
os.makedirs(ckpt_dir, exist_ok=True)
best_model_path = os.path.join(ckpt_dir, "best_model.pkl")

best_accuracy = -1.0
best_params = None

for depth, mode, min_smp, threshold in product(
    param_grid["max_depth"],
    param_grid["mode"],
    param_grid["min_Samples"],
    param_grid["pruning_threshold"]):

    combo_name =
    f"d{depth}_m{mode}_min{min_smp}_th{str(threshold).replace('.', '_')}"
    combo_ckpt = os.path.join(ckpt_dir, f"{combo_name}.pkl")

    print("-----")
    print(f"Combo: max_depth={depth}, mode={mode}, min_Samples={min_smp},
prune_th={threshold}")

    if os.path.exists(combo_ckpt):
        print("Found checkpoint. Loading existing model...")
        tree = joblib.load(combo_ckpt)
    else:
        print("No checkpoint. Training new model...")
        tree = DecisionTree(max_Depth=depth, mode=mode,
                             min_Samples=min_smp,
pruning_threshold=threshold)
        tree.fit(X_train, y_train)
        joblib.dump(tree, combo_ckpt)
        print(f"Saved checkpoint for this combo at {combo_ckpt}")

# ارزیابی
y_pred = tree.predict(X_val)
accuracy = sum(yt == yp for yt, yp in zip(y_val, y_pred)) / len(y_val)
print(f"Validation Accuracy = {accuracy:.4f}")

# به روزرسانی بهترین مدل

```

```

if accuracy > best_accuracy:
    best_accuracy = accuracy
    best_params = {
        "max_depth": depth,
        "mode": mode,
        "min_Samples": min_smp,
        "pruning_threshold": threshold
    }
    joblib.dump(tree, best_model_path)
    print(f"☑ New best model saved! Accuracy = {accuracy:.4f}")

print("=====")
print("Training finished.")
print(f"Best Validation Accuracy = {best_accuracy:.4f}")
print("Best Params:", best_params)

# تست مدل نهایی
if os.path.exists(best_model_path):
    best_tree = joblib.load(best_model_path)
    y_test_pred = best_tree.predict(X_test)
    test_acc = sum(yt == yp for yt, yp in zip(y_test, y_test_pred)) /
len(y_test)
    print(f"Test Accuracy of Best Model = {test_acc:.4f}")

```

نهایتاً کد به این شکل شد.

در مورد مقادیر هایپرپارامتر هایی که قرار داده ام:

با توجه به 54 حالت مختلفی که در نسخه ی قبل تر امتحان کردم متوجه شدم `pruning threshold` وقتی در محدوده ی 0.01 قرار دارد `accuracy` بالاتری برای مدل لحاظ میشود. از این رو در اپدیت جدید مقادیر `pruning threshold` را در این محدوده قرار دادم.

الان مرحله ی توضیح در مورد بخش `data cleaning` :

برای این بخش من از سایت `geeksforgeeks` کمک گرفتم. یعنی مطابق `step` هاش جلو رفتم.

```

# بلاک کد:

=====

==

# DATA CLEANING & PREPROCESSING
#
=====

==

# Load the raw dataset
dataframe = pd.read_csv('EuroRail_Survey.csv')
print('\nDataset Info:')
print(dataframe.info())
print(dataframe.info(), '\n')

#this is to see how many duplicate rows we have in our dataframe:(you will
see there are no duplications.)
print(dataframe[dataframe.duplicated()])

# list comprehension to see the numerical or categorical columns:
cat_col = [col for col in dataframe.columns if dataframe[col].dtype ==
'object']
num_col = [col for col in dataframe.columns if dataframe[col].dtype !=
'object']

print('Categorical columns:', cat_col)
print('Numerical columns:', num_col)

#to see the number of unique values for categorical features:
print(dataframe[cat_col].nunique())

print()
print()

null_percentages = round((dataframe.isnull().sum() / dataframe.shape[0]) *
100, 2)

print(null_percentages)

#
=====

==

# DATA PREPROCESSING PIPELINE

```

```

#
=====

==

# TODO: Apply data cleaning and preprocessing steps

# Step1:Deleting null containing rows:

#As i printed some info about the csv file,the missing datas in all the
columns are less than 0.5%.
# the simplest way to clean this dirty ,is to delete those corresponding
rows:

# this is not needed because we see that all the features have less than
0.5% for null values but i do this working for other csv files too:
np_array_2d = np.column_stack((null_percentages.index,
null_percentages.to_numpy()))

# features array filtered by the percentage of null values less than 0.5%:
values = np_array_2d[:,1].astype(float)
filtered_null_percentage = np_array_2d[values < 0.5, 0]

dataframe = dataframe.dropna(subset=filtered_null_percentage)

# Step2: Calculating Outlier Boundaries and removing them:

for column in num_col:
    mean = dataframe[column].mean()
    std = dataframe[column].std()

    lower_bound = mean - 3 * std
    upper_bound = mean + 3 * std

    dataframe = dataframe[(dataframe[column] >= lower_bound) &
(dataframe[column] <= upper_bound)]

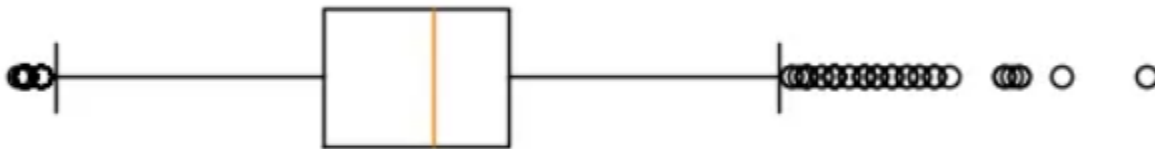
# Step3: Deleting columns that don't carry information for prediction
dataframe = dataframe.drop(columns=['id'])

# Save the cleaned dataset
dataframe.to_csv('EuroRail_Survey_cleaned.csv', index=False)

```

```
print('\nPreprocessing completed successfully!')
print('Cleaned dataset saved and ready for decision tree training.')
```

تا قبل از data preprocessing pipeline همه ی کارهایی که کردم برای debug کردن مشکلات متفاوتی بود. مثلاً یک جایی به اروری برخوردیم که احساس کردم یک ستون categorical احتمالاً یک داده ی numerical داشته باشه، یا اینکه چند تا سطر تکراری داریم و یا اینکه در هر column چند تا null داریم همگی جزو مواردی بود که بررسی کردم. در مورد pipeline مربوط به date cleaning :
کامنت های خلاصه ای که گذاشتم همگی تشریح میکنند که چه کاری انجام میشود.
تنها یک نکته به صورت اسکرین شات در مورد step3 بنظرم ارزش قرار دادن در این گزارش کار را دارد:



این تصویر مربوط به چارک ها هستش که در این datacleaning هم از این موضوع استفاده شد و داده های پرت که در تصویر قابل مشاهده هستش حذف شد از dataset .

نتیجه :

```
Training finished.
Best Validation Accuracy = 0.9158
Best Params: {'max_depth': 12, 'mode': 'gain', 'min_Samples': 50, 'pruning_threshold': 0.01}
Test Accuracy of Best Model = 0.9118
```

رسیدیم به بخش train و test واقعی:

چون برای این بخش نیازه که در اخر visualization انجام بشه ، بهتره که این بخش رو اول بلاک کدش رو قرار بدم :

[illegible]

```

# for leaf show names
if getattr(node, "is_leaf", False):
    ans = getattr(node, "name", None)
    return "EM\\(\\text{ans}\\)\\\\\\(\\text{samples}\\)"

feat = getattr(node, "feature", None)
thr = getattr(node, "threshold", None)
gain = getattr(node, "gain", None)
gini = getattr(node, "gini", None)
lines = []
if feat is not None:
    lines.append("feat: '%feat'")
if thr is not None:
    lines.append("thr: '%thr'")
if gain is not None and not (gain is None):
    # format numeric values shorter
    try:
        lines.append("gain: '%(float(gain), 4)'" )
    except Exception:
        lines.append("gain: '%gain'")
if gini is not None and not (gini is None):
    try:
        lines.append("gini: '%(float(gini), 4)'" )
    except Exception:
        lines.append("gini: '%gini'")
lines.append("samples=\\text{samples}\\")
return "\\join{lines}"

def traverse(node, depth=0, parent_id=None, edge_label=None):
    nid = new_id() # create label. use new_id label to keep maxlines
    lbl = node_summary(node)
    dot_node(nid, label="(%safe_label{lbl})")

    if parent_id is not None:
        is edge_label is None:
            dot_edge(parent_id, nid)
        else:
            dot_edge(parent_id, nid, label=edge_label, fontsize="7") # smaller font size for labels

    # stopping rules: max_display_depth or collapse_threshold
    if max_display_depth is not None and depth > max_display_depth:
        # add a small collapsed marker node (no children)
        collapse_id = new_id()
        samples = 0
        if getattr(node, "datasample_labels", None):
            if os is not None:
                try:
                    samples = len(os)
                except Exception:
                    samples = 0
            dot_node(collapse_id, label="(%safe_label{'... collapsed subtree\\(samples\\)' + str(samples)}))", style="dashed")
            return nid
        # if node has numeric threshold > expected two children (left then right)
        thr = getattr(node, "threshold", None)
        children = getattr(node, ("children", None) or [])

```

```
# if children list contains strings (your implementation sometimes uses children list of values for categorical),
# then in _convert_tree you set children to Node objects. Here we assume they are Node objects.
# handy if children are Node objects (usual case)
for child in children:
    # children might be None, or an object representing a terminal node; handle carefully
    if child is None:
        continue
    # determine edge label (for numeric feature you set edge_value string, else categorical value)
    edge_val = getattr(child, 'edge_value', None)
    traverse(child, depth+1, mid, edge_label+edge_val)
return mid

traverse(tree.root, depth=0, parent_id=None)

# reader to file (filename.pdf)
def _dot_reader(filename, filename.pdf, cleanup=True) # cleanup=True removes intermediary .gv file
    # read file
    with open(filename, 'r') as f:
        return f.read()
    return cleanup
```

این بخش با هماهنگی که با headTA انجام گرفت نیاز نبود شخصا پیاده سازی شود. برای همین از chatgpt برای visualize کردن هرچه بهتر درخت در سه نسخه متفاوت که در فایل زیپ به صورت پی دی اف قرار داده میشود ، کمک گرفتم.

حالا بلاک کد مخصوص train و تست:

```
#
=====
==
# MODEL TRAINING & TESTING
#
=====
==

# Combine the training and validation sets for final training
X_train_final = pd.concat([X_train, X_val])
y_train_final = pd.concat([y_train, y_val])

# Initialize the model with the best hyperparameters found from tuning
# tree = DecisionTree(
#     mode=best_params["mode"],
#     max_Depth=best_params["max_depth"],
#     min_Samples=best_params["min_Samples"],
#     pruning_threshold=best_params["pruning_threshold"]
# )

tree = DecisionTree(
    mode='gain',
    max_Depth=12,
    min_Samples=50,
    pruning_threshold=0.01
)

# Train the model on the combined training data (80%)
tree.fit(X_train_final, y_train_final)

# Evaluate the trained model on the training data itself (for train
accuracy)
train_predict = tree.predict(X_train_final)

# Calculate the final train accuracy
train_accuracy = sum([1 if yt == yp else 0 for yt, yp in
zip(y_train_final, train_predict)]) / len(y_train_final)

# Evaluate the trained model on the unseen test set (20%)
```

```

test_predict = tree.predict(X_test)

# Calculate the final test accuracy
test_accuracy = sum([1 if yt == yp else 0 for yt, yp in zip(y_test,
test_predict)]) / len(y_test)

print(f"Train Accuracy: {train_accuracy:.4f}")
print(f"Test Accuracy (Unseen Data): {test_accuracy:.4f}")
print("\nModel training and testing completed!")

# TODO: Visualize your tree
export_decision_tree_to_pdf(tree, filename="euro_rail_tree_full")

export_decision_tree_to_pdf(tree, filename="euro_rail_tree_depth4",
max_display_depth=4)

export_decision_tree_to_pdf(tree, filename="euro_rail_tree_collapsed",
max_display_depth=6)

print('\nModel training and testing completed!')

```

همانطور که در اول این بلاک کد هم مشخص است من به جای اینکه مقادیر هایپرپارامتر ها رو از **bestparam** که در بخش **hyperparameter tuning** بدست امد استفاده کنم خودم مقدار دهی کرده ام. البته مقادیر هم همان مقادیر **bestparam** هست منتهای دستی اضافه کردم. دلیل این کار من هم به خاطر **limitt خوردن gpu** در **colab** بود که مجبور شدم در یک اکانت دیگر این کار را انجام بدم که طبیعتا نیاز بود هایپرپارامتر تیونینگ رو دوباره انجام بدهم که بسیار وقت گیر بود(در حد یک ساعت و ربع) پس از نتیجه استفاده کردم.(در نوت بوک همه جواب ها سیو شده.)

توضیح کلی که میتوان در مورد این بلاک داد اینه که همانطور که مشخص هستش ، ما یک مدل ترین میکنیم بر اساس هایپرپارامتر های داده شده در مرحله تیونینگ هایپرپارامترها و با داده های **train** (همان 80 درصد)

بعدش تست میکنیم در اون داده های **test** (همان 20 درصد)

و نتایج رو نشان میدیم.

حالا نتایج:

```
Train Accuracy: 0.9274
Test Accuracy (Unseen Data): 0.9161
```

در نوت بوک ذکر شده بود که دلیل رسیدن به این دقت بالای 90٪ را ذکر کنیم.

دلایل رسیدن به این درصد اولاً این هستش که دیتای تمیزی داشتیم و به تبع آن مدل ما (در این پروژه) درخت تصمیم با نویزهای به شدت کمتری train شده است. پس از بروز **overfitting** و یادگرفتن الگوهای نامربوط جلوگیری کردیم. از این رو میتواند داده های **unseen** را به خوبی پیش بینی کند.

نکته ی دیگر رسیدن به این دقت، به موازات دلیل قبلی، تعیین هایپرپارامترهای بهینه بود. همانطور که در مرحله ی تعیین پارامتر هم در این داک نوشتیم با توجه به 54 درختی که تولید شد در **gridsearch** ای که داشتیم، متوجه شدم مقادیر بعضی از هایپرپارامترها را باید نزدیک به برخی مقادیری کنم که باعث افزایش دقت میشود. یعنی یکی دیگر از دلایل رسیدن به این دقت همین بود.

حالا رسیدیم به بخش **post-pruning**:

سعی کردم بعد از نوشتن کد کامنت گذاری کنم تا نیاز به توضیح جزئیات در داک نباشم:

```
# Accuracy function to check the model's performance
def accuracy(x_val, y_val):
    y_pred = tree.predict(x_val) # get predictions
    accuracy = (y_pred == y_val).sum() / len(y_val) # calculate accuracy
    return accuracy

# Main post-pruning function
def postpruning(tree, x_val, y_val):
    initial_accuracy = accuracy(x_val, y_val) # get initial accuracy
    # Nested function to prune nodes
```

```

def node_prune(node):
    nonlocal initial_accuracy
    # Base case: if the node is a leaf, we can't prune further
    if node.is_leaf:
        return

    # Save the current children of the node (we may need to restore them)
    children_track = node.children

    # Temporarily make the node a leaf node by clearing its children
    node.children = []
    node.is_leaf = True

    # Check the accuracy after pruning this node
    current_acc = accuracy(x_val, y_val)

    # If pruning decreased accuracy, revert back
    if current_acc < initial_accuracy:
        node.children = children_track # restore the children
        node.is_leaf = False # restore it to a non-leaf
        for child in node.children:
            node_prune(child) # recurse to prune its children as well
    else:
        # If pruning improved or maintained accuracy, we keep it pruned
        initial_accuracy = current_acc

    # Start pruning from the root node
    node_prune(tree.root)

# Execute post-pruning
postpruning(tree, X_val, y_val)

# Want to see the accuracy on the train dataset after post-pruning:
ytrain_predict = tree.predict(X_train_final)
traindataset_accuracy = (ytrain_predict == y_train_final).sum()/len(y_train_final)

# Want to see the accuracy on the test dataset after post-pruning:
ytest_predict = tree.predict(X_test)
testdataset_accuracy = (ytest_predict == y_test).sum()/len(y_test)

print("The accuracy of the model after post-pruning for train dataset is:",
traindataset_accuracy)
print("The accuracy of the model after post-pruning for test dataset is:",
testdataset_accuracy)

```

```
# Want to see the tree after post-pruning:

# Exporting the tree after post-pruning as a full tree
export_decision_tree_to_pdf(tree, filename="postpruned_full_tree")

# Exporting the tree after post-pruning with a depth limit (depth 4)
export_decision_tree_to_pdf(tree, filename="postpruned_depth_4_tree",
max_display_depth=4)

# Exporting the tree after post-pruning with a collapsed (pruned) version,
showing up to depth 6
export_decision_tree_to_pdf(tree, filename="postpruned_collapsed_tree",
max_display_depth=6)
```

پیاده سازی این تابع به صورت recursive انجام شد. یعنی ما از root درخت شروع میکنیم. چک میکنیم که اگر بچه هایش حذف شوند در این صورت آیا accuracy ما در داده های validation بیشتر میشود یا نه. اگر افزایش نیافت کاری انجام نمیدهیم. اگر که افزایش یافت ما این نود را leafnode میکنیم. البته برای root که کاملاً مشخص هست این اتفاق (افزایش دقت validation) اتفاق نمی افتد از این رو من توضیحات را برای یک نود فرضی در یک جایی از درخت در نظر بگیرید: بچه های نود رو فعلاً حذف شده در نظر میگیریم. دقت درخت را در داده های validation میسنجیم با این فرض که آن نود بچه ای ندارد (یعنی از آن نود به بعد هرس شده است). اگر افزایش یافته بود یعنی کار درستی کرده بودیم و نهایتاً به عنوان leafnode قرار میگیرد. اگر افزایش نیافته بود، بچه های این نود رو برمیگردانیم. یعنی هرسی رخ نمیدهد. این بررسی abstract این recursion بود. یعنی این کار از leafnode میشود و هی callback میخورد تا برسد به root node. در نهایت ما درختی خواهیم داشت که overfit ندارد. یعنی این هرس ما باعث شده overfit در آن کاهش یابد. یعنی یک گام دیگر در مسیر کاهش overfit.

نتیجه این بخش:

```
The accuracy of the model after post-pruning for train dataset is: 0.9256742092818715
The accuracy of the model after post-pruning for test dataset is: 0.9167085484534199
```

کاهش هزارم درصدی دقت مدل در داده های train

افزایش ده هزارم درصدی دقت مدل در داده های تست.

این یعنی اینکه داده های نويز حذف شده.الگویی که یاد گرفته مدل مون بهتر شده بعد هرس.

در نتیجه الان model ای که train شده جزو مدل های قوی است.

در نهایت درختی که از این بخش به دست امد در فایل zip قرار داده شد.

پایان!